

Bern2Edge: A Neurosymbolic Compiler for Edge Deployment via Bernstein Polynomial Networks

Abstract—Deploying high-accuracy neural networks on resource-constrained edge devices remains challenging, as existing approaches treat training, compression, and hardware synthesis as separate stages, leaving a gap between software-trained models and efficient end-to-end deployment with limited support for interpretability. We propose Bern2Edge, an end-to-end framework that uses knowledge distillation to convert a pretrained teacher feed-forward network into hardware-efficient representations via Bernstein polynomial activations. This representation enables two deployment paths: (i) a high-fidelity LUT-based realization that preserves model fidelity under compression, and (ii) a symbolic rule-based representation derived from Bernstein activation geometry, enabling interpretable inference with explicit input-space constraints. The resulting BNNs achieve up to 2.12 percentage point accuracy improvement over ReLU under identical compression constraints. At the system level, Bern2Edge achieves up to 99.8% latency reduction and 95% BRAM reduction relative to a W8A8 quantized teacher on an AMD Xilinx KV260 FPGA, while maintaining accuracy within 0.5 pp, and further deploys on a low-power Spartan-7 XC7S15 FPGA. The rule-based path reduces DSP usage by up to 89.0% at a cost of 1.5 pp in total accuracy.

Index Terms—Bernstein polynomials, activation functions, knowledge distillation, lookup table, FPGA inference, symbolic rule extraction, edge ML.

I. INTRODUCTION

The rapid growth of deep neural networks (DNNs) has driven significant gains in predictive performance, but has increased the difficulty of deploying these models on resource-constrained edge devices. Modern networks are typically trained in software without accounting for hardware constraints, yielding models that are inefficient to deploy and hard to map to strict latency and resource budgets. Their limited interpretability further restricts use in edge settings where transparency and reliability are critical. Bridging high-level model design and efficient hardware realization remains a key challenge, as existing approaches treat model design and hardware optimization separately, limiting their ability to satisfy both accuracy and system-level constraints [1].

Existing work addresses parts of this problem through model compression, quantization, and hardware compilation frameworks such as hls4ml [2], FINN [3], and CGRA4ML [4]. These translate trained networks into synthesizable hardware, often applying quantization and pruning during deployment. However, they operate on fixed neural representations, mapping or optimizing pre-trained models for hardware rather than jointly designing representations inherently aligned with hardware efficiency.

In this work, we introduce **Bern2Edge**, an end-to-end edge deployment framework based on Bernstein polynomial activations [5], targeting feed-forward networks (FFNs), including

multilayer perceptrons (MLPs) and the FFN sublayers of transformers. The structured, bounded representation of Bernstein polynomials suits both hardware realization and symbolic reasoning [6]. Building on this, our pipeline transforms a trained DNN teacher into hardware-efficient representations for synthesis. First, knowledge distillation (KD) [7] trains a compact student with Bernstein activations, a Bernstein Neural Network (BNN). This enables two deployment paths: a lookup table (LUT)-based realization, where a per-neuron LUT represents the Bernstein activation for compact, efficient synthesis; and an optional symbolic rule-based representation, derived from the geometric structure of Bernstein activations, enabling further compression and interpretable inference. To our knowledge, this is the first work to jointly exploit the structural properties of Bernstein polynomial activations for both efficient hardware synthesis and symbolic rule extraction.

The contributions of this work are as follows:

- **End-to-end teacher-to-hardware deployment framework.** We propose **Bern2Edge**, a unified pipeline that compresses a trained teacher DNN into a hardware-efficient BNN via KD. The resulting representation supports two deployment paths targeting different objectives: a LUT-based realization for efficient hardware execution, and a symbolic rule-based representation for additional compression and interpretable inference.
- **Bernstein activations for improved compression under knowledge distillation.** We introduce Bernstein polynomial activations as a structured student representation that improves knowledge distillation under strong compression. BNNs recover the accuracy of large DNN teachers more effectively than standard activations such as ReLU under compressed regimes, while remaining well-suited for hardware realization and interpretability.
- **LUT-based hardware realization.** We show that Bernstein polynomial activations admit a direct LUT-based implementation that leverages their normalized input domain and fixed functional structure. This enables efficient per-neuron LUT realization, providing a compact representation for edge inference.
- **Symbolic rule extraction and synthesis.** We develop a BNN specific rule extraction method that derives compact symbolic rules from the geometric structure of learned Bernstein activations. The resulting geometry-aligned rules form a structured, hardware-efficient representation that supports quantization to low bitwidth with negligible accuracy loss.
- **Evaluation in compression and deployment regimes.** We evaluate **Bern2Edge** on multiple tabular datasets and a transformer FFN setting, demonstrating: (i) improved performance of BNNs under strong compression, (ii)

efficient LUT-based hardware synthesis with cost comparable to or less than standard activations, (iii) compact and effective symbolic rule representations, (iv) end-to-end deployment on AMD Xilinx KV260 and low-power Spartan-7 XC7S15 FPGAs, and (v) robustness of the rule network under input noise and distribution shift.

II. BACKGROUND AND RELATED WORK

A. Edge Deployment Frameworks and Model Compression

A large body of work targets neural network deployment on resource-constrained hardware through compiler frameworks and model compression. *hls4ml* [2] translates trained networks into FPGA implementations via high-level synthesis, mapping dense and convolutional operators to streaming architectures with fixed-point quantization. *FINN* [3] extends this to quantized networks, generating dataflow accelerators for low-bitwidth, high-throughput FPGA inference. *CGRA4ML* [4] extends these approaches to coarse-grained reconfigurable architectures (CGRAs). While such systems improve portability and performance, they primarily map and optimize pre-trained models for hardware through post-training transformations such as quantization, pruning, and operator fusion, rather than co-designing the model representation to align with hardware constraints.

Our work shifts the focus from post-training compression to *training-time* representation: we train Bernstein polynomial activations whose learned forms are inherently deployable, rather than adapting arbitrary activations for hardware after the fact. This enables direct symbolic extraction and a unified pipeline from training to hardware implementation.

Since many high-performing models are pretrained with ReLU-based architectures, we employ knowledge distillation (KD) to transfer large pretrained teachers into compact BNNs. KD is a standard compression technique that matches soft outputs or intermediate representations between a teacher and student [7]. In **Bern2Edge**, we use it to train student BNNs that retain the accuracy of much larger models while maintaining a structured, hardware-friendly representation.

B. Hardware Realization of Nonlinear Activations

Efficient implementation of nonlinear activations remains a key challenge in hardware deployment. Piecewise-linear functions such as ReLU map efficiently to comparator-based logic, but expressive smooth activations such as GeLU [8] and Swish [9] require more complex arithmetic and are typically approximated using lookup tables or piecewise-polynomial methods [10], introducing a train-deploy gap between the trained and deployed function.

LUT-centric designs address this by mapping neuron computations directly to lookup tables. LUTNet [11] uses native FPGA LUTs as inference operators to reduce arithmetic cost, while LogicNets [12] co-designs sparse, quantized neurons extractable as LUT truth tables. Polynomial-based methods such as PolyLUT [13] and PolyLUT-Add [14] extend this to structured polynomial classes, but both require hard fan-in limits of ≤ 7 inputs per neuron to bound LUT size: PolyLUT

enforces sparse connectivity, while PolyLUT-Add sums low-fan-in sub-neurons. This constraint is incompatible with the dense tabular MLPs we target (14–54 input features), requiring additional sparsification or redesign to apply. **Bern2Edge** instead retains dense MLP layers and makes the learned activation itself hardware-realizable.

Recent work explores activation-centric LUT mappings. KANELE [15] uses Kolmogorov–Arnold Networks (KANs), replacing MLP computations with learnable one-dimensional edge functions discretized into per-neuron LUTs, departing from standard MLP structure and requiring per-edge evaluations. In contrast, **Bern2Edge** retains standard FFN architectures and targets the activation directly: Bernstein activation coefficients define the deployed function, enabling exact realization via small per-neuron LUTs without a train-deploy gap.

C. Bernstein Polynomial Activations

• **Bernstein Polynomials.** A Bernstein polynomial $\sigma(z)$ of degree n , defined on the interval $[l, u]$, can be written as [6]:

$$\sigma(z; l, u, \mathbf{c}) = \sum_{k=0}^n c_k b_{n,k}(t), \quad t = \frac{z-l}{u-l} \in [0, 1], \quad z \in [l, u], \quad (1)$$

where z is the input to the polynomial function, t is the normalized input, and $\mathbf{c} = \{c_0, \dots, c_n\}$ are the polynomial coefficients controlling the activation shape. The Bernstein basis functions are defined as:

$$b_{n,k}(t) = \binom{n}{k} t^k (1-t)^{n-k}, \quad k = 0, \dots, n, \quad (2)$$

which satisfy the partition of unity property:

$$\sum_{k=0}^n b_{n,k}(t) = 1, \quad \forall t \in [0, 1]. \quad (3)$$

This representation defines the polynomial as a convex combination of its coefficients over a bounded domain, with the coefficients directly controlling the activation shape.

• **Bernstein Neural Networks (BNNs).** Bernstein polynomials have been explored in neural networks as a nonlinear activation function due to their favorable analytic and approximation properties. Khedr et al. [5] introduce Deep Bernstein Networks, feed-forward neural networks in which standard activation functions are replaced by learnable Bernstein polynomials. We adopt this formulation throughout **Bern2Edge** and refer to it as a Bernstein Neural Network (BNN). For a network of depth L , the input is denoted as $\mathbf{x}^{(0)}$ and the output of the l -th layer as $\mathbf{x}^{(l)}$. The propagation rule is given by:

$$\mathbf{x}^{(l)} = \sigma(\mathbf{z}^{(l)}; \mathbf{c}^{(l)}), \quad \mathbf{z}_i^{(l)} = \left(\mathbf{w}_i^{(l)}\right)^\top \mathbf{x}^{(l-1)} + \beta_i^{(l)} \quad (4)$$

where $\mathbf{w}_i^{(l)}$ and $\beta_i^{(l)}$ are the learnable weights and biases. The activation function σ operates element-wise, parametrized by a set of learnable Bernstein coefficients $\mathbf{c}^{(l)} = \{c_k^{(l)}\}_{k=0}^n$. Here, n is a hyperparameter for the polynomial degree. This formulation allows the network to learn the shape of its nonlinearities alongside its weights. For simplicity of notation, we drop the superscript (l) in the remainder of the paper.

• **Theoretical Properties of BNNs.** (1) Stable training: Using polynomials as activation functions in deep NNs has attracted several researchers’ attention in recent years, though a major drawback is their unstable training behavior due to exploding gradients, which becomes more prominent with increasing degree [16]. Bernstein polynomials avoid this limitation, keeping the resulting BNN free of this instability [5], [17]. (2) Exponential approximation rates and parameter efficiency: Bernstein-based architectures also achieve approximation error bounds that are superior to standard feed-forward ReLU networks [17]. ReLU networks are piecewise-linear approximators that require a large number of neurons to accurately approximate a smooth, nonlinear function, whereas Bernstein polynomials are inherently suited to approximating smooth functions at much lower complexity. These properties motivate the use of BNNs to compress larger networks.

In **Bern2Edge**, we exploit the structure of the Bernstein polynomial activation for both hardware synthesis and symbolic rule extraction.

D. Symbolic Rule Extraction

Prior work on symbolic rule extraction from neural networks falls into pedagogical and compositional methods. Pedagogical approaches such as TREPAN [18] treat the trained network as a black box and induce a decision tree by querying its input–output behavior. Instead, compositional approaches exploit internal network structure. DeepRED [19] extends earlier rule extraction methods to deep networks, extracting intermediate rules layer by layer, and using decision trees to describe hidden-layer behavior. More recent work such as ECLAIRE [20] improves the scalability of compositional rule extraction while maintaining high-quality, interpretable rule sets. Expanding on this, recent frameworks like NeuSymHLS [21] employ partial symbolic distillation by leveraging hardware-aware symbolic regression to replace internal neural layers with compact analytic expressions.

Earlier activation-aware techniques, such as validity interval analysis [22], derive symbolic descriptions by propagating activation intervals through the network or projecting internal activation constraints back to the input space. These methods incorporate activation information but rely on numerical interval propagation and treat activations generically, without exploiting the analytic structure of a learned activation family.

Our method in **Bern2Edge** differs from these approaches by deriving rule extraction directly from the analytic geometry of the learned activation. Instead of inducing decision trees or propagating interval bounds, we derive rule candidates from the geometric structure of the learned Bernstein activations. This yields semantically aligned partitions tied to the network’s learned nonlinear representation and supports a compact symbolic rule set suitable for efficient deployment.

III. BERN2EDGE FRAMEWORK OVERVIEW

We propose **Bern2Edge**, an end-to-end pipeline that transforms a trained teacher model into hardware-efficient and interpretable representations. The framework consists of four

modules: (1) BNN training via KD, followed by two alternative deployment paths: (2) LUT-based hardware realization, or (3) symbolic rule extraction followed by (4) hardware synthesis and deployment. Fig. 1 shows the **Bern2Edge** pipeline. The following sections describe each module and the Bernstein polynomial properties that enable these capabilities.

A. BNN Training via Knowledge Distillation

Knowledge Distillation Setup. The first step in the **Bern2Edge** pipeline is to train compressed BNNs as student models using KD [7], reducing the model size while preserving accuracy.

Bernstein-based networks exhibit favorable approximation properties under a constrained parameter budget, enabling accurate function representation with fewer parameters than standard activations [17]. This property motivates our use of Bernstein activations in the student network: a Bernstein-based student can recover more of the teacher’s accuracy than a ReLU-based student of equal size, an effect we confirm across all evaluated architectures in Section IV.

This behavior is further reflected in the BNN’s learned decision geometry: Bernstein-based models can capture smoother and more curved decision boundaries, while standard activations such as ReLU tend to produce piecewise-linear boundaries, as illustrated in Fig. 2.

The student objective follows the standard distillation loss [7]:

$$\mathcal{L}_s = (1 - \alpha)\mathcal{L}_{CE} + \alpha T^2 \text{KL}(S(\mathbf{y}^*/T) \| S(\hat{\mathbf{y}}/T)), \quad (5)$$

where $\mathcal{L}_{CE}$ is the cross-entropy loss between student predictions and ground-truth labels, $S(\cdot)$ is the softmax, $\text{KL}(\cdot \| \cdot)$ is the Kullback–Leibler divergence between softened teacher and student distributions, and $\hat{\mathbf{y}}$ and $\mathbf{y}^*$ are the student and teacher logits, with $\hat{\mathbf{y}} = \mathbf{x}^{(L)}$ the student’s final layer output. T is the temperature controlling the softness of the output distribution, and α balances supervision between hard labels and teacher guidance.

Teacher models are selected to provide strong supervision signals consistent with prior tabular benchmarks. Configurations follow MLP baselines reported in prior work [23]. Student model architectures are designed to satisfy strict compression and latency constraints. While Bernstein activations introduce additional learnable parameters in the form of polynomial coefficients, these do not translate to increased hardware cost under our LUT-based activation implementation as described in the next module of **Bern2Edge** (Section III-B). **BNN Fixed-Bound Training.** Bernstein activations operate over a fixed input domain, requiring inputs to be normalized to $[0, 1]$ as defined in Eq. 1. This fixed-domain property is later leveraged for efficient hardware realization. After a short warmup phase, per-neuron pre-activation bounds $[l_i, u_i]$ are calibrated from empirical activation statistics and then fixed for the remainder of training. Calibration is performed sequentially, one layer at a time, once preceding layers have stabilized.

Fixing these bounds serves two purposes: it stabilizes training by preventing continual shifts in activation normalization,

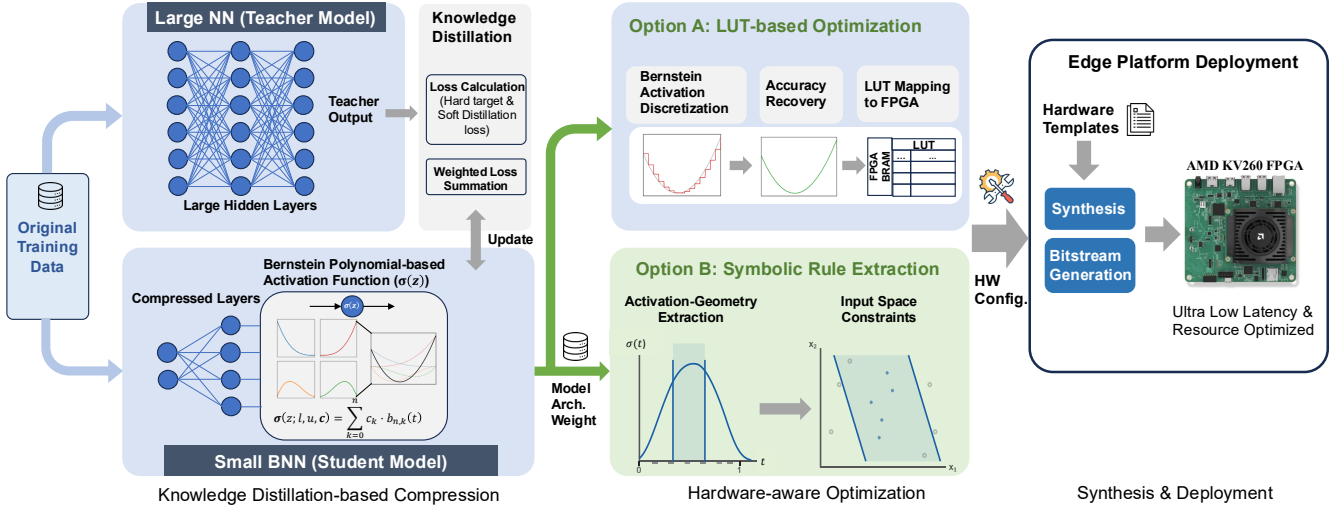


Fig. 1: Overview of **Bern2Edge**: A high-accuracy teacher model is distilled into a compressed BNN student via KD. The resulting representation is synthesized and deployed via either exact LUT-based realization or symbolic rule extraction.

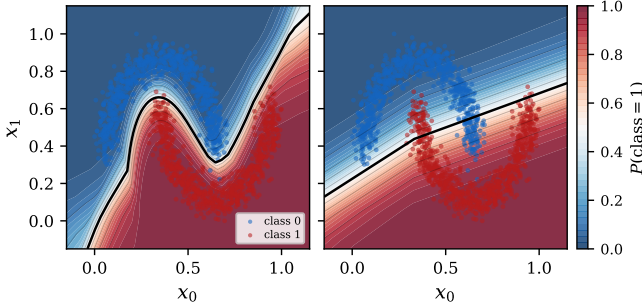


Fig. 2: Decision boundaries learned by a BNN (left) and a ReLU model (right) of equal capacity on the synthetic Two Moons dataset [24]. The BNN captures the curved structure of the data, while the ReLU model produces a simpler, predominantly linear boundary.

and it enforces the fixed-domain consistency required for exact LUT-based deployment, as discussed in Section III-B.

To discourage out-of-bounds inputs to the activation function, we add an out-of-bounds penalty in the BNN loss function:

$$\mathcal{L}_{\text{BNN}} = \mathcal{L}_s + \lambda_{\text{ob}} \cdot \mathbb{E} [\max(0, -t) + \max(0, t - 1)], \quad (6)$$

where t denotes the normalized pre-activation, as defined in Eq. 1. During forward propagation, activations are clamped after normalization to preserve the Bernstein basis structure, ensuring non-negativity and partition-of-unity as in Eq. 3. Our evaluation in Section IV-B will compare compressed student BNNs via KD against similarly compressed ReLU networks under identical hardware constraints.

B. LUT-Based Hardware Realization

The next stage of the **Bern2Edge** framework is the hardware synthesis of the trained BNN on FPGA. As shown in Fig. 1,

one deployment path leverages a LUT-based realization of Bernstein activations.

Direct evaluation of the Bernstein activation in Eq. 1 is computationally expensive. For a polynomial of degree n , it requires $(n + 1)$ power operations, $(n + 1)$ multiplications with binomial coefficients, and an $(n + 1)$ -term dot product with learned coefficients, resulting in approximately $3(n + 1)$ multiply-accumulate (MAC) operations per neuron. However, since Bernstein activations operate over a fixed normalized domain $[0, 1]$, the activation function can be precomputed. Each neuron's activation is evaluated offline over a uniform grid of E points in $[0, 1]$ and stored in on-chip memory (BRAM). At runtime, activation evaluation reduces to a single memory access, eliminating all arithmetic operations.

BRAM storage scales linearly with the number of neurons h in a layer and grid resolution E . While storage grows with h , the complete removal of activation-phase computation yields substantial savings in DSP usage and combinational logic. This trade-off motivates training of compact BNN students using KD (Section III-A), keeping h small enough for efficient on-chip storage.

A naive implementation uses nearest-neighbor lookup via floor indexing, which quantizes inputs into E discrete levels and introduces an approximation error. To mitigate this, we employ low-cost linear interpolation between adjacent LUT entries:

$$f(t) = \mathcal{T}[i_{\text{lo}}] + \mu \cdot (\mathcal{T}[i_{\text{hi}}] - \mathcal{T}[i_{\text{lo}}]), \quad (7)$$

where t is the normalized pre-activation as defined in Eq. 1, $\mathcal{T}[\cdot]$ is the E -entry LUT, $i_{\text{lo}}, i_{\text{hi}}$ are the indices of the two adjacent grid points bracketing t , and $\mu \in [0, 1]$ is the fractional position between them. This interpolation significantly improves accuracy while introducing minimal overhead: one multiply, one subtraction, and one addition per neuron, all

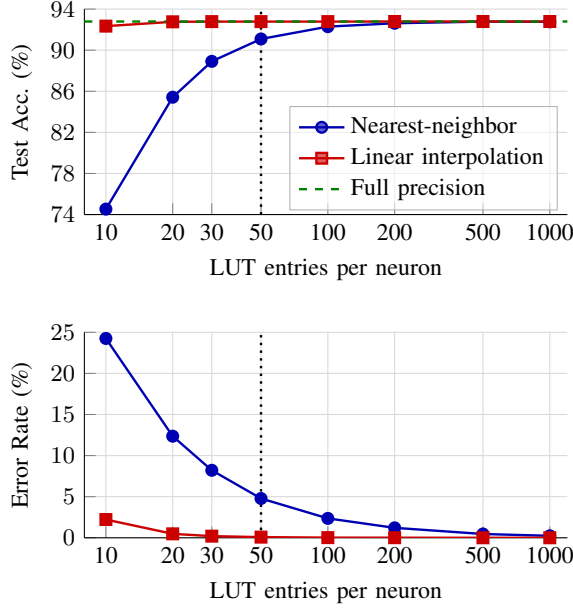


Fig. 3: Per-neuron LUT behavior vs. table size on Coverttype: test accuracy (top) and error rate (bottom) vs. LUT entries per neuron. Error% is the fraction of samples where LUT output differs from full-precision inference.

implemented in combinational logic with no additional DSP or BRAM usage.

Empirically, for a BNN with hidden layer size $h = 256$ on Coverttype [25], linear interpolation at 50 entries (100 KB BRAM) achieves an error rate of 0.08%, roughly 60 $\times$ lower than nearest-neighbor’s 4.78% at the identical table size and storage cost. We select 50 entries as our operating point for the remainder of this work. Nearest-neighbor error falls off slowly with table size and does not reach linear interpolation’s 50-entry error rate even at 1000 entries, showing that interpolation reaches low error at a fraction of the storage nearest-neighbor would require. Fig. 3 plots accuracy and error rate against table size for both methods on this setting, showing the rapid saturation of linear interpolation against the persistently slower convergence of nearest-neighbor.

Finally, to eliminate additional runtime overhead, the input normalization required by Eq. 1: $t = (z - l_i)/(u_i - l_i)$ is fused into the preceding linear layer’s weights offline:

$$W_{\text{fused}}[i][j] = W[i][j]/(u_i - l_i) \quad (8)$$

$$\beta_{\text{fused}}[i] = (\beta[i] - l_i)/(u_i - l_i) \quad (9)$$

The linear layer output then arrives already normalized to $[0, 1]$, so the activation requires only a clamp and an index computation, improving hardware efficiency. This fusion is only possible because Bernstein activations use fixed, learned input bounds l_i and u_i that do not change at runtime.

C. Symbolic Rule Extraction

The second path to deployment in **Bern2Edge** begins with symbolic rule extraction, as shown in Fig. 1. We propose an activation-geometry-based pipeline that converts the trained student BNN into a compact, interpretable rule set.

Coefficient-driven structure and analytic regime partitioning: The coefficients $\mathbf{c}$ fully determine the shape of Bernstein activation over the normalized domain. The derivative preserves this structure through differences between adjacent coefficients:

$$\sigma'(z) = \frac{n}{u-l} \sum_{k=0}^{n-1} (c_{k+1} - c_k) b_{n-1,k} \left(\frac{z-l}{u-l} \right). \quad (10)$$

The roots of $\sigma'(z)$ define local extrema that partition the activation into monotonic regimes determined directly by the learned coefficients. In the **Bern2Edge** framework, these regimes encode the learned activation geometry and form the basis for symbolic rule extraction and subsequent hardware synthesis.

This formulation is specific to BNNs, whose coefficient-driven polynomial structure enables analytic extraction of multiple activation regimes from the learned function. In contrast, piecewise-linear activations such as ReLU define only a single breakpoint and lack the internal coefficient structure needed to support multiple intrinsic regimes, making them less amenable to this extraction method.

Pipeline of symbolic rule extraction: The rule extraction pipeline in **Bern2Edge** consists of three stages. Stage 1 extracts neuron-wise activation regimes from the Bernstein activation geometry in pre-activation space. Stage 2 composes these regimes into candidate rules and selects a compact subset using a cascade greedy cover based on purity and coverage. Stage 3 optimizes the selected rules for deployment via weight vector sparsification and quantization, producing a hardware-efficient final rule set. We detail each stage below.

1) *Stage 1: Geometry Extraction:* The first stage extracts neuron-wise activation regimes directly from the Bernstein activation geometry in pre-activation space (z -space) as follows.

- **Pre-activation mapping.** Each neuron i defines a scalar pre-activation (simplified from Eq. 4):

$$z_i = \mathbf{w}_i^\top \mathbf{x} + \beta_i, \quad (11)$$

which maps to a normalized domain $t \in [0, 1]$ using learned input bounds $[l_i, u_i]$ (Section III-A).

- **Activation motif classification.** The learned Bernstein coefficients $\mathbf{c}_i$ fully determine the shape of each activation. Based on these coefficients, we classify neurons into qualitative *motifs* (e.g., monotone, bump, valley) that describe the overall structure of the activation response, as shown in Fig. 4.

- **Regime boundary extraction.** We compute regime boundaries analytically from the activation geometry. Breakpoints in t -space come from three sources: (i) a uniform grid that ensures baseline coverage, (ii) roots of the first derivative identifying local extrema, and (iii) roots of the second derivative identifying inflection points. As Eq. 10 shows, these derivatives take closed form in terms of coefficient differences, enabling analytic computation of all breakpoints without dependence on data samples. The resulting breakpoints reflect the learned activation shapes, as illustrated in Fig. 4.

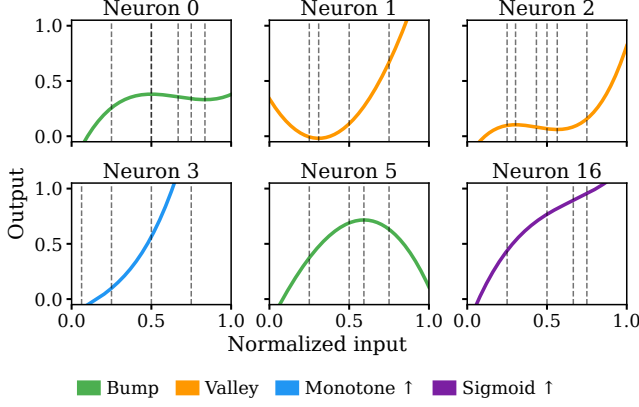


Fig. 4: Bernstein activation curves with analytically derived regime breakpoints. Six representative neuron activations from a BNN ($h = 64$) trained on Adult [26], with breakpoints from derivative roots and inflection points marked.

- **Input-space mapping.** We map the breakpoints back to pre-activation space using the fixed input bounds $[l_i, u_i]$, via

$$z_i = l_i + t(u_i - l_i), \quad (12)$$

the inverse of the normalization in Eq. 1, producing disjoint intervals that partition each neuron’s response into regimes determined entirely by its coefficients. The convex hull property of Bernstein polynomials [6] bounds the activation output by the range of its coefficients; when restricted to subintervals, the effective bounds become tighter, further reinforcing the stability of the resulting regime partitions.

The mapping of these regimes to input-space constraints is illustrated step-by-step in Fig. 5. We apply this mapping to first-layer neurons, whose pre-activations are affine in the raw input $\mathbf{x}$ (Eq. 11); a selected interval in t -space defines a corresponding interval in z -space via Eq. 12, which, through Eq. 11, directly induces an affine constraint in input space of the form

$$z_i^{\text{lo}} \leq \mathbf{w}_i^\top \mathbf{x} + \beta_i \leq z_i^{\text{hi}}, \quad (13)$$

defining an oblique band in input space. Each condition in a rule corresponds to one neuron and one of its activation regimes; the conjunction of conditions across multiple neurons produces intersecting affine bands, forming oblique polyhedral regions in the input space. Stage 2 constructs candidate rules from these intervals and refines them based on empirical quality.

2) Stage 2: Candidate Rule Generation and Selection:

Given the regime partitions from Stage 1, we define rules as conjunctions of affine constraints of the form in Eq. 13, one per neuron regime. We assign each rule the majority predicted class of its covered samples.

- **Rule evaluation.** We evaluate each candidate rule r on training data by computing the set of samples satisfying all its conditions, from which we compute: (i) *purity*, the fraction of covered samples whose prediction matches

the rule’s label, and (ii) *coverage*, the number of samples matched by the rule. We discard rules that do not meet minimum coverage and purity thresholds; both are hyperparameters of the method.

- **Candidate generation.** We construct rules progressively by combining regimes across neurons, beginning with single-condition rules evaluated for all neurons and their associated regimes. Rules that do not meet the desired purity become *impure seeds*, which we iteratively expand by adding conditions from additional neurons up to a predefined depth limit.

At each expansion step, we evaluate candidate extensions formed by adding one condition to each impure parent rule. Among all valid extensions, we retain two: (i) the highest-coverage extension that meets the purity criterion, and (ii) the highest-coverage extension that does not, which serves as the seed for further expansion. This retains at most one pure and one impure extension per parent rule, bounding candidate growth as the number of neurons and regimes increases while preserving the highest-coverage candidates.

- **Cascade greedy selection.** From the candidate pool, we select a compact rule set via a cascade greedy cover procedure. The cascade proceeds over decreasing purity thresholds with increasing minimum coverage requirements, ensuring that lower-purity rules are selected only if they add substantial new coverage.

Within each stage, we select rules iteratively using a scoring function that prioritizes covering new samples while penalizing redundancy and conflicts:

$$\text{score}(r) = \text{gain}(r) - \alpha_{\text{sc}} \cdot \text{same-cover}(r) - \alpha_{\text{conf}} \cdot \text{conf}(r). \quad (14)$$

Here, $\text{gain}(r)$ is the number of samples covered by r that are not yet covered by the current rule set. The term $\text{same-cover}(r)$ counts overlap with selected rules of the same label, while $\text{conf}(r)$ counts overlap with rules of different labels. The weights α_{sc} and α_{conf} control the corresponding penalties; [their effect on coverage and accuracy is analyzed in Section IV-F.](#)

At each step, we select the rule with the highest score and sufficient new coverage and update coverage accordingly. The process continues until no further rules can be added, after which the algorithm proceeds to the next lower purity stage until the minimum purity threshold is reached.

This cascade yields a compact rule set that first captures high-confidence regions and then expands coverage while controlling redundancy and conflicts. The resulting rule set provides a geometry-aligned approximation of the model’s decision boundary (Fig. 6).

- 3) **Stage 3: Rule Optimization:** To enable efficient inference on resource-constrained hardware, we optimize the extracted rules for reduced computation and memory usage as follows.

- **Rule sparsification.** Each rule condition takes the affine form of Eq. 13. In practice, a small subset of large-magnitude weights often dominates the output, indicating that only a few input features contribute significantly. We

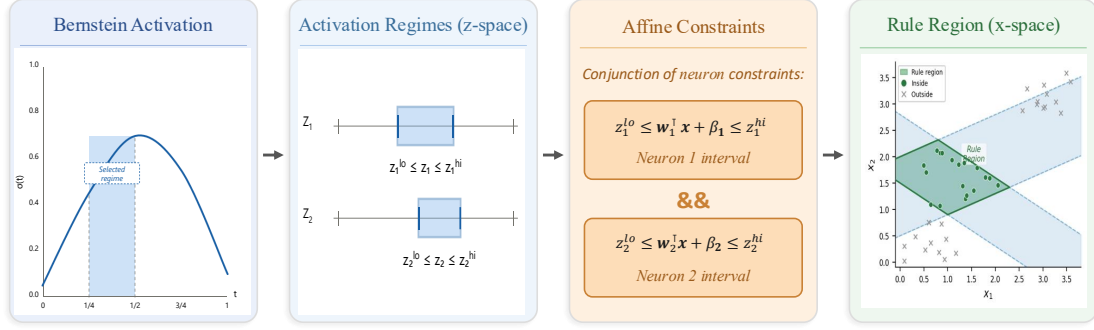


Fig. 5: Activation-geometry-based rule formation. A selected regime in normalized activation space (t) maps through z -space to an affine constraint in input space, forming interpretable oblique regions.

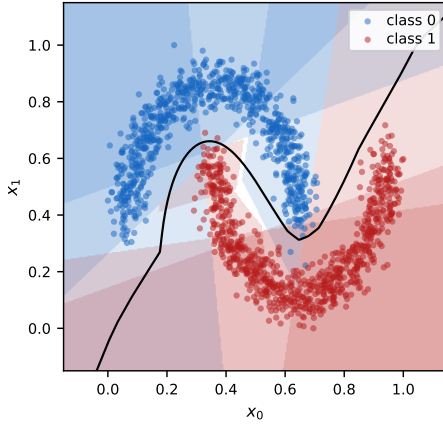


Fig. 6: Activation-geometry rule regions on the Two Moons dataset [24]. Rules align with the curved decision boundary via Bernstein-derived breakpoints, enabling compact partitioning.

therefore retain only the top- k entries of each weight vector and set the remaining entries to zero, reducing both storage and inference cost to k multiplications per condition. k is a hyperparameter of the method; its effect on memory footprint and accuracy is ablated in Section IV-F.

- **Integer quantization.** We quantize rule parameters to integer precision, enabling direct deployment on integer-only hardware. Weight vectors use symmetric per-vector INT8 quantization:

$$\mathbf{w}_q = \text{round}\left(\frac{\mathbf{w}}{s}\right), \quad s = \frac{\max_j |w_j|}{127},$$

and we dequantize as $\hat{\mathbf{w}} = s \cdot \mathbf{w}_q$. We quantize thresholds and biases to fixed-point precision.

Unlike multi-layer neural networks, quantization here does not introduce cross-layer error accumulation, since each rule evaluates as a single affine operation on the original input. As a result, INT8 quantization introduces negligible degradation in rule accuracy, with an average accuracy drop of 0.002% across all tested datasets.

The final rule set consists of sparse, quantized constraints evaluable with integer and fixed-point arithmetic, enabling hardware-efficient inference.

4) *Inference:* The final stage is to perform rule inference as follows.

- **Conflict resolution.** Due to overlapping regimes, a sample may satisfy multiple rules with different labels. We resolve conflicts by selecting the matching rule with the highest training purity:

$$\hat{y} = \arg \max_{r \in \mathcal{R}(\mathbf{x})} \text{purity}(r), \quad (15)$$

where $\mathcal{R}(\mathbf{x})$ is the set of rules whose conditions are all satisfied by sample $\mathbf{x}$, and $\text{purity}(r)$ is the purity of rule r measured on the training set (Stage 2).

- **Fallback.** Inputs not matched by any rule fall through to a Classification and Regression Tree (CART) trained on the uncovered training samples for which no rule fires [27]. The CART partitions the feature space via threshold comparisons on raw features, requiring no multiplications. We keep the tree shallow (depth 4) to preserve interpretability and minimize hardware cost. Like the rules, we quantize the tree's split thresholds to fixed-point `fix<16, 8>` before synthesis. We compare CART against alternative fallback strategies (linear regression, a small BNN trained on uncovered samples, and the full underlying BNN) in Section IV-F (Table IX).

5) *Rule Synthesis:* We synthesize each rule set into hardware operating in three pipelined phases. Phase 1 computes per-condition linear projections as sparse INT8 dot products: each condition stores only non-zero quantized weights alongside their feature indices in on-chip ROM, reducing multiply operations and weight storage relative to dense evaluation. We fully unroll and pipeline the dot product, dequantizing the result by a per-condition scale factor. Phase 2 evaluates each rule by checking whether its condition projections fall within learned band boundaries (lower/upper thresholds); the highest-purity firing rule determines the predicted label. Phase 3 routes samples not matched by any rule to the CART fallback. All rule parameters reside as compile-time ROM constants, requiring no external memory access.

IV. RESULTS

A. Experimental Setup

• **Datasets and Preprocessing.** We evaluate **Bern2Edge** on tabular benchmarks from prior work [23], and a language classification task for the transformer generalization study. In particular, **HIGGS-Small** [28], [29] (98,049 samples, 28 features, binary), **Covertypes** [25] (581,012 samples, 54 features, 7 classes), and **Adult Census** [26] (48,842 samples, 14 mixed features, binary). These datasets use standard preprocessing and stratified splits following Gorishniy et al. [23]. **MAGIC Gamma Telescope** [30] (19,020 samples, 10 features, binary) is used for rule extraction benchmarking and rule certification. **ACS Income** [31], a published covariate-shift benchmark, evaluates rule-system robustness under geographic and temporal distribution shift. **SST-2** [32] evaluates Bernstein activations in transformer FFN sublayers, with TinyBERT4 [33] as the target architecture. All experiments use 5-fold cross-validation; test data are transformed using preprocessors fitted on the training split.

• **Teacher and Student Models.** Teacher models replicate the exact MLP configurations of Gorishniy et al. [23], achieving accuracy within 0.5 percentage points (pp) of reported baselines. Students are trained via KD [7] using identical architectures with either Bernstein activations (degree 3 for MLPs; degree 15 for transformer FFN sublayers) or ReLU. Training uses AdamW with learning rate 6×10^{-3} , weight decay 1×10^{-4} , and cosine annealing; BNNs additionally include an out-of-bounds penalty ($\lambda_{\text{ob}} = 10^{-2}$, Section III-A). Distillation parameters are swept per dataset with temperature $T \in \{2, 4\}$ and $\alpha \in [0, 0.85]$, and the best-performing configuration is reported. Student architectures are selected by sweeping width and depth under hardware constraints. We denote architectures as $\{d_{\text{in}}, h_1, \dots, h_m, d_{\text{out}}\}$; where a single hidden layer is varied, we write its width as a scalar h .

• **Hardware Setup.** Training uses an NVIDIA Tesla V100 (16GB). Designs are synthesized with Vitis HLS 2024.1 and Vivado 2024.1 and evaluated on an AMD Xilinx KV260 FPGA at 200 MHz. We also evaluate on a Spartan-7 XC7S15 FPGA to demonstrate low-power edge deployment.

• **Evaluation.** We evaluate three components: (1) neural compression, comparing Bernstein and ReLU students under identical constraints; (2) symbolic rule extraction; and (3) the full **Bern2Edge** pipeline against a W8A8 (8-bit weight, 8-bit activation) quantization-aware training (QAT) teacher baseline [34]. We further conduct three additional studies: low-power deployment on the Spartan-7 XC7S15, ablation on the rule extraction hyperparameters (k , α_{sc} , α_{conf}) and fallback strategies, and rule robustness evaluation covering input-noise certification and distribution-shift assessment on benchmarks.

• **Metrics.** We report test accuracy and cross-entropy loss. For hardware deployment, we report latency, DSPs, BRAM, LUTs, and flip-flops (FFs). For rule extraction, we additionally report rule count, average conditions per rule, and coverage.

B. Student BNN Compression and Synthesis

Table I reports accuracy and synthesis metrics for BNNs and equivalent ReLU student networks across

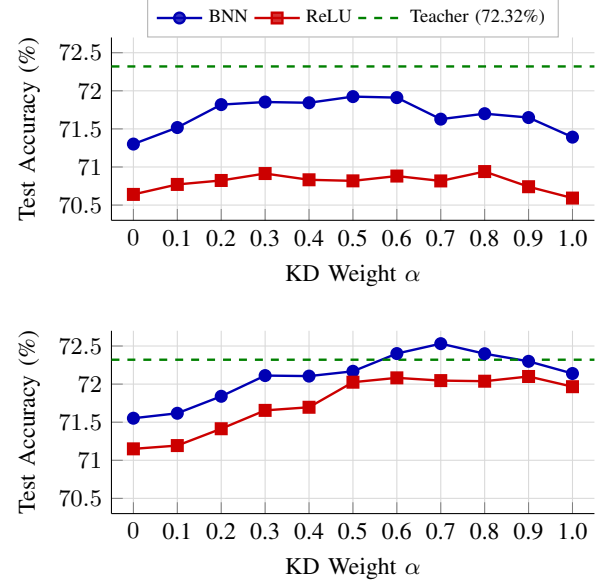


Fig. 7: Test accuracy vs. KD weight α ($T=2$) on HIGGS-Small for students with single layer $h = 16$ (top) and $h = 128$ (bottom). Dashed line: teacher baseline (72.32%).

three datasets. Teacher models follow the MLP baselines of Gorishniy et al. [23]: $\{28, 231, 121, 2\}$ on HIGGS-Small, $\{54, 1024, 1024, 512, 7\}$ on Covertypes, and $\{14, 503, 503, 503, 111, 2\}$ on Adult.

• **Accuracy Under Compression.** BNNs consistently outperform their ReLU counterparts across all architecture–dataset combinations (Table I), with accuracy gains of +0.25 to +2.12 pp and lower cross-entropy loss in 8 of 9 cases. The gap is largest under strong compression, reaching +2.12 pp on Covertypes $\{54, 64, 32, 7\}$, and narrows as model capacity increases, consistent with the approximation behavior of Bernstein-based networks [17]. Notably, BNNs approach or exceed teacher accuracy at smaller architectures than ReLU models on every dataset. Although gains on Adult and HIGGS-Small are modest, they are consistent across all folds; inter-model differences below one percentage point are typical for state-of-the-art tabular models [23], indicating that the improvements reflect genuine learning efficiency rather than incidental variation.

• **Alignment with Teacher under KD.** Fig. 7 shows test accuracy vs. α on HIGGS-Small for a small student [16] (panel a) and a large student [128] (panel b). In the small student, the BNN maintains a stable ~ 1 pp lead over the ReLU model across all α values, with neither activation reaching the teacher; model width is the binding constraint, and the Bernstein advantage persists even without distillation ($\alpha=0$). In the larger student, both activations improve with α , but the BNN crosses the teacher near $\alpha=0.7$ and peaks at 72.5%, while the ReLU model approaches but does not reach the teacher. Together, the two panels show that at low capacity the Bernstein advantage is intrinsic and KD-invariant, while at higher capacity the BNN additionally benefits more from stronger distillation signals.

• **Hardware Overhead.** Bernstein activations impose modest and predictable hardware overheads relative to ReLU at

TABLE I: Accuracy and synthesis metrics on AMD KV260 (5-fold CV). Latency in clock cycles. $\Delta\text{Acc} / \Delta\text{CE}$: BNN minus ReLU.

Dataset	Arch.	Act.	Acc. (%)	CE Loss	$\Delta\text{Acc} / \Delta\text{CE}$	Latency	DSPs	BRAMs	LUTs
HIGGS-Small	{28, 16, 2}	ReLU	70.98 $\pm$ 0.20	0.553	+0.94 / -0.013	85	108	0	5397
		Bern	71.92 $\pm$ 0.26	0.540		87	106	2	5682
	{28, 16, 8, 2}	ReLU	71.84 $\pm$ 0.24	0.540	+0.48 / -0.008	121	156	0	6852
		Bern	72.32 $\pm$ 0.18	0.532		125	135	3	7294
	{28, 128, 2}	ReLU	72.08 $\pm$ 0.27	0.524	+0.45 / -0.012	337	108	34	5114
		Bern	72.53 $\pm$ 0.18	0.512		339	103	44	5410
Coverttype	{54, 64, 32, 7}	ReLU	88.97 $\pm$ 0.82	0.292	+2.12 / -0.069	960	360	113	20213
		Bern	91.09 $\pm$ 0.27	0.223		963	326	105	21815
	{54, 128, 64, 7}	ReLU	93.53 $\pm$ 0.18	0.138	+1.52 / -0.060	2477	360	137	22207
		Bern	95.05 $\pm$ 0.05	0.078		2481	326	136	23824
	{54, 256, 128, 7}	ReLU	95.62 $\pm$ 0.12	0.057	+0.77 / -0.029	7429	360	169	22432
		Bern	96.39 $\pm$ 0.02	0.028		7433	326	179	23063
Adult	{14, 16, 2}	ReLU	84.06 $\pm$ 0.39	0.324	+0.54 / -0.010	68	69	0	3823
		Bern	84.60 $\pm$ 0.13	0.314		70	69	2	4113
	{14, 32, 16, 2}	ReLU	84.48 $\pm$ 0.12	0.315	+0.44 / -0.009	161	163	1	7739
		Bern	84.92 $\pm$ 0.13	0.306		165	136	6	8132
	{14, 128, 2}	ReLU	84.57 $\pm$ 0.22	0.310	+0.25 / +0.001	321	67	20	3595
		Bern	84.82 $\pm$ 0.14	0.311		323	61	30	3892

TABLE II: Coverttype accuracy under matched latency and BRAM budgets. Parentheses: actual synthesis values (cycles, BRAMs).

Latency (cyc)	BRAMs	Bernstein Acc (%) (cyc, BRAM)	ReLU Acc (%) (cyc, BRAM)	ΔAcc (pp)
≤ 200	≤ 30	76.50 $\pm$ 0.44 (189, 26)	74.93 $\pm$ 0.08 (188, 25)	+1.57
≤ 400	≤ 55	82.98 $\pm$ 0.16 (397, 52)	81.87 $\pm$ 0.37 (396, 49)	+1.11
≤ 1000	≤ 120	91.09 $\pm$ 0.27 (963, 105)	88.97 $\pm$ 0.82 (960, 113)	+2.12
≤ 2500	≤ 140	95.05 $\pm$ 0.05 (2481, 136)	93.53 $\pm$ 0.18 (2477, 137)	+1.52
≤ 7500	≤ 180	96.39 $\pm$ 0.02 (7433, 179)	95.62 $\pm$ 0.12 (7429, 169)	+0.77

matched architectures (Table I). Latency overhead does not exceed 4 clock cycles in any configuration, as each per-neuron Bernstein evaluation reduces to a single LUT lookup followed by linear interpolation. Bernstein uses fewer or equal DSPs than ReLU at every architecture, with savings reaching 27 on Adult {14, 32, 16, 2}. The compiler performs bit-width optimization, pruning unused upper bits from the activation output, so each downstream multiply fits in 2 DSPs instead of 3. ReLU, being unbounded, preserves the full fixed-point $\text{fix}\langle 32, 16 \rangle$ range and requires 3 DSPs per multiply. LUT counts are higher for Bernstein across all configurations (5–8% overhead), reflecting the synthesized per-neuron lookup tables; BRAM overhead is small and architecture-dependent, governed by the interaction between per-neuron table size and BRAM tile granularity.

• **Accuracy Under Matched Hardware Budgets.** Table II selects the best Bernstein and ReLU models on Coverttype within five hardware budget tiers, confirming that Bernstein outperforms ReLU under identical tight constraints at every operating point. Gains range from +0.77 pp at the 7500-cycle tier to +2.12 pp at 1000 cycles, ruling out parameter count and architecture size as confounding factors.

• **Polynomial Degree and Hardware.** Fig. 8 compares FPGA resource utilization for naive Bernstein evaluation, which directly computes Eq. 1, against our LUT-based implementation across degrees $n \in \{3, 5, 7, 9, 11\}$. Naive evaluation scales

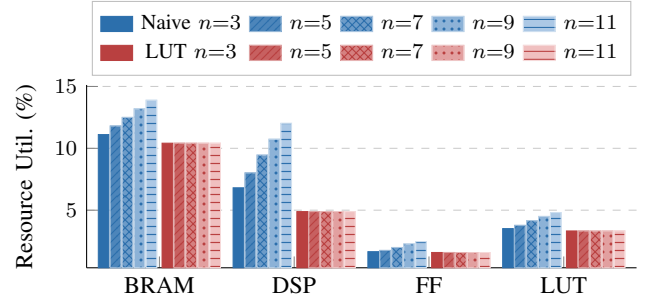


Fig. 8: FPGA resource utilization on Adult ($h = 128$) for naive Bernstein evaluation and LUT-based inference across polynomial degrees $n \in \{3, 5, 7, 9, 11\}$.

with degree because each additional basis function requires more multiply-accumulate hardware, causing DSP and LUT usage to grow monotonically. In contrast, our implementation has constant resource cost: every activation maps to the same fixed-size lookup table at inference time. This decouples degree selection from hardware budget, allowing n to increase for harder approximation tasks [17] without additional FPGA overhead.

C. Symbolic Extraction Results

• **Rule Extraction on Adult.** Rules are extracted from BNNs trained with $T=2$ and $\alpha=0.5$ on Adult. The extraction pipeline (Section III-C) uses purity threshold 0.85, minimum coverage of 5 samples per rule, maximum depth 3, and sparsity $k=7$. We sweep α_{sc} and α_{conf} to study the coverage–compactness trade-off. An additional evaluation on MAGIC compares against published baselines [20].

Table III reports rule extraction results across five BNN architectures on Adult. Covered accuracy (84.5–87.2%) consistently exceeds each model’s test accuracy (84.33–84.63%), confirming the extracted rules are at least as discriminative as the underlying network on covered samples. α_{sc} controls a clear coverage–compactness trade-off: at $\alpha_{sc}=0.5$, compact

TABLE III: Rule extraction on Adult across five BNN architectures. α_{conf} : conflict penalty constant = 0.1. CART fallback. α_{sc} : same-coverage penalty; $\bar{\ell}$: avg. conditions per rule; Cov.: coverage; Cov. Acc.: covered accuracy; Acc_t : total accuracy.

Arch	BNN Acc. (%)	α_{sc}	Rules	$\bar{\ell}$	Cov. (%)	Cov. Acc. (%)	Acc_t (%)
{14, 16, 2}	84.33	0.5	37	1.76	78.2	86.8	82.6
		0.1	45	1.67	91.5	84.5	81.7
{14, 32, 2}	84.42	0.5	45	2.04	84.9	87.2	83.2
		0.1	53	2.02	94.0	85.3	82.1
{14, 128, 2}	84.63	0.5	61	1.64	89.5	85.9	82.8
		0.1	85	1.75	95.8	85.3	82.0
{14, 16, 8, 2}	84.33	0.5	29	1.69	84.0	86.9	83.1
		0.1	39	1.77	89.9	86.3	82.3
{14, 32, 16, 2}	84.36	0.5	55	2.02	77.6	87.2	82.8
		0.1	66	1.95	91.3	85.1	81.8

TABLE IV: LUT-based BNNs vs. rule networks HW results on Adult (AMD KV260).

Arch	Method	Acc (%)	Latency	DSP	BRAM	LUT	FF
{14, 16, 2}	LUT	84.33	77	77	2	4,329	5,582
	Rules	82.59	183	30	5	2,997	2,290
{14, 32, 2}	LUT	84.42	119	109	4	6,019	8,333
	Rules	83.16	215	30	8	3,009	2,218
{14, 128, 2}	LUT	84.63	376	301	28	15,609	22,865
	Rules	82.76	279	30	8	3,002	2,217
{14, 16, 8, 2}	LUT	84.33	108	96	3	5,715	6,912
	Rules	83.12	151	30	5	3,011	2,300
{14, 32, 16, 2}	LUT	84.36	172	144	6	8,352	10,990
	Rules	82.78	255	30	8	3,027	2,218

rule sets cover 77.6–89.5% of test samples with covered accuracy 85.9–87.2%; at $\alpha_{\text{sc}}=0.1$, coverage rises to 89.9–95.8%, but with more rules and slightly lower covered accuracy (84.5–86.3%). Total accuracy Acc_t remains within 1.2–2.6 pp of the underlying BNN, with the gap widening at lower coverage settings where more samples fall to the CART fallback. Sensitivity analysis of α_{sc} , α_{conf} , and k as interpretable trade-off knobs, along with an ablation of fallback strategies, are provided in Section IV-F.

• **Hardware Comparisons.** Table IV compares LUT-based BNNs against rule networks on hardware. The rule network consumes fixed resources (30 DSPs) regardless of architecture, while LUT networks scale with width, reaching 301 DSPs at {14, 128, 2}. At this largest architecture, the rule network reduces DSP count by 10 $\times$ and latency by 1.35 $\times$, at a cost of 1.87 pp in total accuracy. At smaller architectures, DSP savings persist but rule-based latency exceeds LUT inference, as the fixed rule-matching overhead dominates. Rule extraction thus trades a modest accuracy cost for interpretability: predictions are expressed as conjunctions over original input features, enabling direct inspection of decision logic without hardware overhead scaling with model size.

• **Comparison to Prior Methods.** Table V compares symbolic rule extraction on MAGIC against published baselines [20], evaluated under 5-fold cross-validation. We use ECLAIRE’s reference ReLU MLP {10, 64, 32, 16, 2} as the teacher and evaluate two settings: (i) rule extraction from a BNN trained from scratch on the same architecture (no KD), and (ii) rule

TABLE V: Rule extraction on MAGIC Gamma Telescope. Baselines from Zarlenga et al. [20] (mean $\pm$ std, 5 folds); Decompositional: rule extraction from intermediate layers; $\bar{\ell}$: avg. conditions per rule; Fid: fidelity to baseline network.

Type	Method	Rules	$\bar{\ell}$	Acc (%)	Fid (%)
Decompositional	DeepRED [19]	5143 $\pm$ 9799	5.43	78.7	89.3
	REM-D [35]	3617 $\pm$ 6748	5.41	78.6	89.4
	ECLAIRE [20]	396 $\pm$ 75	3.82	84.6	89.4
Bern2Edge	no-KD ($\alpha_{\text{sc}}=0.1$)	46 $\pm$ 4	1.84	84.2	91.8
	no-KD ($\alpha_{\text{sc}}=0.5$)	24 $\pm$ 3	1.81	82.9	94.3
	KD ($\alpha_{\text{sc}}=0.1$)	44 $\pm$ 4	1.86	84.4	91.8
	KD ($\alpha_{\text{sc}}=0.5$)	24 $\pm$ 4	1.87	82.8	94.6

extraction from a compressed BNN student {10, 64, 32, 2} distilled from that teacher ($T=2$, $\alpha=0.5$).

The extraction pipeline uses a purity threshold of 0.85, sparsity $k=3$, and maximum rule depth of 2. The KD configuration matches ECLAIRE’s accuracy (84.4% vs. 84.6%) with 9 $\times$ fewer rules (44 $\pm$ 4 vs. 396 $\pm$ 75) and half the average rule complexity (1.86 vs. 3.82 conditions), while achieving higher fidelity to the underlying network (91.8% vs. 89.4%). Without KD, rule sets remain compact (24–46 rules) at comparable accuracy (82.9–84.2%). This indicates rule compactness is primarily attributable to Bernstein activation geometry rather than distillation, with KD contributing a marginal additional accuracy and fidelity gain.

D. Bern2Edge End-to-End Results

Table VI presents end-to-end post-synthesis results for **Bern2Edge** against the quantized teacher baseline across all three datasets. **Bern2Edge** (LUT) reduces latency by 97.3–99.8% relative to the W8A8 teacher while matching or staying within 0.5 pp of teacher accuracy. DSP savings reach 27.4% on HIGGS-Small and 74.7% on Adult. On Covtype, the apparent BRAM increase (128 $\rightarrow$ 149) reflects a memory-architecture shift: the teacher relies on 52 URAM tiles for weight storage, whereas **Bern2Edge** substitutes standard BRAM and eliminates 94.2% of URAM usage (52 $\rightarrow$ 3), reducing DSPs by 16.2%.

On Adult, the rule deployment path cuts DSP usage to 30 ($\downarrow$ 89.0%) relative to the teacher, at a cost of 1.5 pp in total accuracy compared to the LUT path. Beyond resource savings, rule deployment constrains predictions to explicit conjunctions over the original input features, providing formal guarantees on the input subspace covered by each decision; a property unavailable in either the teacher model or the LUT-based BNN.

E. Deployment and Optimization on Low-Power Edge

We evaluate both **Bern2Edge** deployment paths on severely resource-constrained hardware. The Spartan-7 XC7S15 [36], a low-power FPGA, provides only 8k LUTs, 20 DSPs, and 20 BRAM18K at $\sim$ 16 mW static power, 14.6 $\times$, 62 $\times$, and 14.4 $\times$ fewer LUTs, DSPs, and BRAMs than the KV260, respectively. We deploy LUT-based BNNs on Adult using the {14, h , 2} architecture over $h \in \{4, 8, 16, 32, 64, 128\}$ hidden layer sizes, and symbolic rule networks, across two rule-count configurations. Two device-level optimizations bring

TABLE VI: Post-synthesis accuracy and end-to-end hardware results for **Bern2Edge** across three datasets. Percentages indicate reduction relative to the W8A8 teacher baseline. Rule deployment path reported for Adult only.

Method	Acc (%)	Latency (cycles)	DSPs	BRAMs	URAMs
<i>HIGGS-Small</i>					
Teacher (W8A8)	72.3	6,981	186	50	—
Bern2Edge (LUT)	72.3	125 (↓ 98.2%)	135 (↓ 27.4%)	3 (↓ 94.0%)	—
<i>Covertypes</i>					
Teacher (W8A8)	96.9	91,634	389	128	52
Bern2Edge (LUT)	96.4	2,481 (↓ 97.3%)	326 (↓ 16.2%)	149 (↑ 16.4%)	3 (↓ 94.2%)
<i>Adult</i>					
Teacher (W8A8)	84.6	40,305	273	42	20
Bern2Edge (LUT)	84.6	70 (↓ 99.8%)	69 (↓ 74.7%)	2 (↓ 95.2%)	—
Bern2Edge (Rules)	83.12	151 (↓ 99.6%)	30 (↓ 89.0%)	5 (↓ 88.1%)	—

TABLE VII: Deployment on XC7S15. R50 and R29 denote the 50-rule and 29-rule symbolic classifiers, respectively. Acc is test accuracy; Acc_{fp} the fp32 reference for BNNs.

Model	Config.	LUT	FF	DSP	BRAM 18K	Lat. (cyc)	Pwr (mW)	Acc (%)	Acc _{fp} (%)
Rules	R50	1,086	844	16	6	340	62	83.02	—
	R29	710	765	16	4	156	59	83.01	—
Bern. {14, h, 2}	h=4	553	883	18	1	61	51	84.18	84.25
	h=8	635	804	19	1	82	63	84.57	84.64
	h=16	662	777	19	1	115	63	84.88	84.86
	h=32	667	799	19	2	179	58	84.92	84.92
	h=64	698	826	19	5	308	60	84.82	84.92
	h=128	858	870	19	13	565	66	84.95	85.03

each path within budget without altering any model parameters. For the rule system, *distributed-RAM rule storage* binds the 3.5KB INT8 rule ROM to LUTRAM rather than block RAM, bringing it within the BRAM budget. For the BNN, *fixed-point quantization* reduces the linear-layer word from $\text{fix}\langle 32, 16 \rangle$ to $\text{fix}\langle 18, 8 \rangle$, mapping each linear-layer multiply to a single DSP rather than three, bringing every configuration within the DSP budget at a cost of ≤ 0.1 pp accuracy. The bitwidth split reflects two separate constraints: 8 integer bits are required to represent Adult’s categorical codes (up to 40), while the remaining 10 fractional bits bound quantization rounding error and recover accuracy to within 0.1 pp of fp32 at no added DSP cost. Table VII reports post-synthesis results across all configurations. Every design fits comfortably within device limits, and DSP count stays nearly flat across all widths due to the time-multiplexed $\Pi=1$ linear-layer datapath, so wider networks pay primarily in latency and activation BRAM rather than combinational logic. The rule-based classifiers achieve the smallest overall footprint, confirming that symbolic deployment is the resource-optimal path when interpretability is required.

F. Hyperparameter and Fallback Ablation

Bern2Edge exposes three parameters governing rule extraction: the sparsity threshold k , the redundancy penalty α_{sc} , and the conflict penalty α_{conf} . Each controls a specific hardware–accuracy trade-off with monotone, predictable behavior. The rule purity threshold is set close to the underlying network’s

TABLE VIII: Joint penalty sweep on Adult ($h = 32$, BNN acc. 84.42%). Conf.: conflicting samples; Cov.: coverage; Cov. Acc.: covered accuracy; Acc_t: total accuracy; $\bar{\ell}$: avg. conditions per rule.

α_{conf}	α_{sc}	Conf.	Cov. (%)	Cov. Acc. (%)	Acc _t (%)	Rules	$\bar{\ell}$
0.1	0.1	308	94.01	85.26	82.10	53	2.02
1.0	0.1	156	92.60	84.67	82.39	50	2.06
0.1	0.3	136	87.15	86.72	83.16	53	2.06
1.0	0.3	111	86.99	86.77	83.16	52	2.04
0.1	0.5	121	84.86	87.18	83.16	45	2.04
1.0	0.5	106	84.42	87.18	83.39	46	2.09
0.1	1.0	78	82.40	87.23	83.43	37	1.95
1.0	1.0	70	80.69	87.61	83.46	36	1.94

accuracy, ensuring extracted rules are at least as discriminative as the model they summarize.

• **Sparsity threshold k** k controls per-rule memory footprint only, with no effect on rule count. Fig. 10 shows BRAM grows linearly with k while total accuracy plateaus beyond $k=7$ with increasing variance for $k>7$. We select $k=7$, achieving $\sim 55\%$ of the dense memory footprint ($k=14$) at no accuracy cost. This plateau reflects that most rule weights are small and can be zeroed without loss; sparse rules match dense accuracy because the dropped conditions carry little discriminative weight.

• **Penalty parameters** Table VIII details the full metric picture (conflict count, rule count, and average conditions per rule) across joint $(\alpha_{sc}, \alpha_{conf})$ settings. Fig. 9 isolates each parameter’s effect on coverage and covered accuracy. α_{sc} is the primary lever: increasing it from 0 to 1 trades coverage ($\sim 100\% \rightarrow \sim 70\%$) for covered accuracy ($\sim 79\% \rightarrow \sim 84\%$), with an elbow near $\alpha_{sc} \approx 0.2$. This is a deployment choice, not a sensitivity: broad-coverage applications favor small α_{sc} ; high-confidence rule firing favors large α_{sc} . Within each α_{sc} regime, α_{conf} reduces raw conflict count (Table VIII) but has negligible effect on coverage or covered accuracy ($\lesssim 0.6$ pp variation across the full range), since conflicts are resolved at inference time by firing the highest-purity matching rule. α_{conf} therefore requires no careful tuning.

• **Fallback Ablation** Table IX compares four fallback strategies on the Adult dataset across accuracy, fidelity to the underlying network on uncovered samples, and hardware cost. The linear regression fallback operates in first-layer pre-activation space $\mathbf{z} = W_0 \mathbf{x} + \beta_0$, folded into an equivalent input-space classifier $\mathbf{w}_{eff}^T \mathbf{x} + \beta_{eff}$. The small BNN fallback uses a one-hidden-layer network of width 4 trained on uncovered samples only; the full BNN is the original network, so its fidelity is 100% by definition. We select CART (depth ≤ 4) as the fallback: it achieves the best overall accuracy (83.02%) and the highest accuracy on uncovered samples (66.1%) at the lowest hardware cost (714 LUT, 471 FF, 0 DSP). Notably, even the full BNN matches neither metric, suggesting uncovered samples are inherently difficult to classify regardless of model capacity. The full BNN also requires $2.6\times$ more LUTs and $4.9\times$ more flip-flops than CART. Linear regression offers the lowest latency (13 cycles) but yields the worst accuracy on uncovered samples (61.3%), reflecting the limitations of a linear decision boundary in this region.

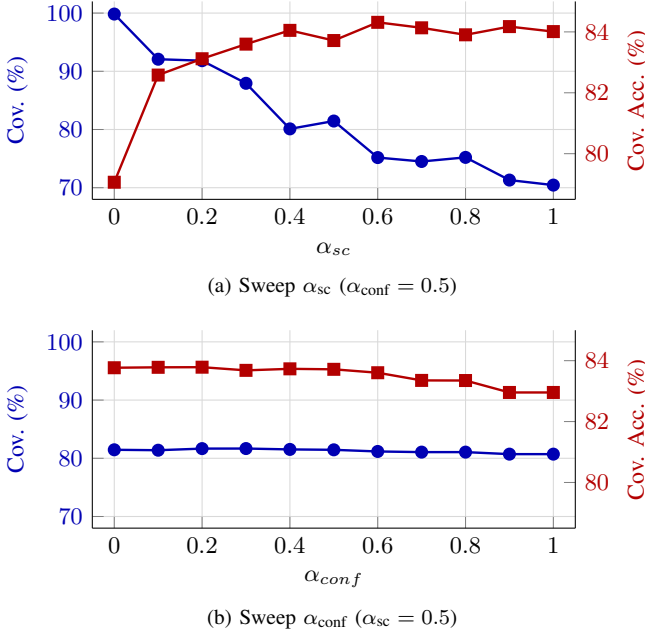


Fig. 9: Coverage and covered accuracy as a function of the two rule extraction penalty parameters on Adult ($h = 32$).

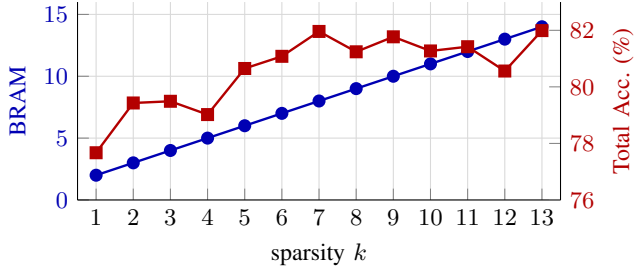


Fig. 10: BRAM and total accuracy vs. sparsity threshold k on Adult ($h = 32$).

G. Symbolic Rule Certification and Robustness

• **Rule Certification on Input Noise.** We evaluate the symbolic rule system on MAGIC, using a $\{10, 64, 32, 2\}$ architecture for the ReLU network and BNN, and the rules extracted from that BNN. Inputs are perturbed by bounded per-feature noise with budget $\epsilon_i = c \cdot s_i$, where s_i is the training-set standard deviation of feature i . The range $c \leq 0.05$ corresponds to sub-5% relative uncertainty, consistent with calibrated process measurement devices [37]; $c = 0.10$ serves as a stress test beyond normal operating conditions.

For each rule slab $z^{\text{lo}} \leq \mathbf{w}^\top \mathbf{x} < z^{\text{hi}}$, the raw input box $[x_i - \epsilon_i, x_i + \epsilon_i]$ is mapped through the monotone quantile normalization, and the exact worst-case interval of $\mathbf{w}^\top \mathbf{x}$ is propagated over the box. A point is *rule certified* if no different-label rule of equal or higher purity is reachable within the box, i.e. no admissible perturbation can change the predicted label. This certificate is conservative and therefore never falsely certifies stability.

We compare rule certificates against network-level certificates for the underlying BNN and equivalent ReLU network. The BNN uses Bern-IBP with de Casteljau subdivision [5]; the ReLU network uses standard IBP via auto_LiRPA [38],

TABLE IX: Fallback strategy ablation on the Adult dataset ($h = 32$, $\alpha_{sc} = 0.5$, $\alpha_{conf} = 0.1$). Resource columns report the fallback component only. Acc_{unc} and Fid_{unc} are on uncovered samples. All accuracy and fidelity values in %.

Fallback	Acc.	Acc_{unc}	Fid_{unc}	LUT	FF	DSP	Lat.
Linear Regression	81.67	61.3	84.1	1749	520	10	13
Full BNN	82.91	65.8	100.0	1887	2318	40	68
Small BNN	82.23	65.2	90.5	1129	2178	38	46
CART	83.02	66.1	87.7	714	471	0	26

TABLE X: Robustness certification on MAGIC. Empirical columns show rule accuracy and rule-BNN fidelity under sampled noise; certified columns show provably stable fractions for rules and networks.

c	Empirical (%)		Certified (%)		
	Rule Acc.	Fid.	Rules	BNN	ReLU
0.00	100.0	98.2	100.0	100.0	100.0
0.01	99.6	97.8	83.4	82.1	80.1
0.03	99.0	97.2	59.4	41.3	27.1
0.05	98.4	96.4	42.7	16.0	12.4
0.10	97.0	94.7	22.7	6.6	4.8

following the comparison methodology of Khedr et al. [5]. Empirical robustness is estimated with 20 i.i.d. uniform perturbations per point from the same noise box ($\approx 50,000$ perturbed inputs per noise level). Since uniform sampling rarely reaches box corners, empirical robustness is an upper bound; certified columns are sound lower bounds.

Table X reports results on the covered-correct test subset (2,485 of MAGIC test points). Empirical rule accuracy drops by only 3.0 pp at $c = 0.10$, and rule-BNN fidelity remains above 95% for all realistic noise levels ($c \leq 0.05$). The certified results reveal a larger separation: at $c = 0.05$, rule certification reaches 42.7%, versus 16.0% for the BNN and 12.4% for the matched ReLU network. This advantage stems from certifying compact linear slab rules directly; the resulting intervals are tighter than those produced by network-level IBP at every noise level tested. Among the neural models, the BNN achieves higher certified stability than the matched ReLU network at every noise level, consistent with the tighter bounds produced by Bern-IBP [5].

• **Rule Robustness to Data Distribution Shift.** A key concern for streaming edge deployment is whether rules remain reliable under distribution shift. Table XI evaluates this on ACS Income [31], training on CA-2018 and testing under temporal and geographic shifts. The ReLU teacher uses a $\{10, 512, 256, 128, 2\}$ architecture; the ReLU and BNN students both use $\{10, 32, 2\}$, with rules extracted from the BNN student.

Under temporal shift, rule coverage is essentially stable (+0.2 pp) and covered accuracy drops by only 1.6 pp, closely tracking the underlying BNN’s own 1.1 pp accuracy loss. Geographic shift is harder: coverage falls 4.3 pp but remains above 85%, and covered accuracy drops 3.9 pp, comparable to the 6.4–6.9 pp accuracy degradation of the neural baselines. In both conditions, symbolic extraction does not amplify the

underlying model’s degradation. Crucially, uncovered samples are explicitly flagged at deployment and routed to the fallback model, so distributional failures surface as measurable coverage drops rather than silent mispredictions.

H. Extending to Transformer FFN Layers

We extend Bern2Edge to the FFN sublayers of TinyBERT4 [33], demonstrating BNN deployment in transformer architectures. The original TinyBERT4 FFN hidden size ($h=1200$) is compressed to $h \in \{312, 600\}$ across all FFN layers, with all other components unchanged. Training follows three stages: (1) function-matching from the teacher GeLU FFN to a degree-15 BNN at the target width, where higher degree imposes no additional hardware cost under LUT-based synthesis (Fig. 8), (2) substitution of the compressed BNN into the original TinyBERT4 transformer, and (3) 10 epochs of KD fine-tuning from the original TinyBERT4 using the training scheme in Section III-A.

We synthesize and implement all TinyBERT4 encoder variants on KV260. The $h=600$ Bernstein FFN matches or exceeds TinyBERT4 accuracy (90.48% vs. 90.37%) despite halving the FFN hidden width. The Bernstein variants’ reduced computational depth translates directly to latency, with end-to-end cycle count dropping by up to 61.0% relative to the teacher and the FFN sublayer alone reaching 72.2% (Table XII). It also reduces DSP, FF, and LUT usage relative to both the teacher and the GeLU baseline at matched width, whereas GeLU’s direct polynomial computation remains at the teacher’s DSP count regardless of compression. The higher BRAM usage of Bernstein FFNs relative to GeLU is attributable to per-neuron LUT storage, though both remain below TinyBERT4’s original footprint. Despite their smaller size, both compressed GeLU variants increase FF usage over the teacher, as their buffers are synthesized as distributed RAM, implemented directly in FFs and LUTs rather than dedicated memory blocks.

V. LIMITATIONS AND FUTURE WORK

Bern2Edge exposes three rule extraction parameters (k , α_{sc} , and α_{conf}), each with a monotone, predictable effect on a specific hardware–accuracy trade-off (Section IV-F). A natural next step is automated design-space exploration (DSE) that sweeps these knobs to select the Pareto-optimal configuration for a given deployment constraint.

Although Bernstein activations are architecturally compatible with transformer FFN sublayers, rule extraction over dense latent representations limits rule semantics. Concept probing [39] identifies semantically labeled directions in the hidden space, which could enable rule extraction over a compact, interpretable basis.

Bernstein activations are not inherently limited to tabular MLPs or transformer FFNs. Polynomial networks have demonstrated superior representation power over ReLU-based CNNs on vision tasks [40], making convolutional extension of **Bern2Edge**’s pipeline a natural future direction.

VI. CONCLUSION

We presented **Bern2Edge**, a neurosymbolic compiler for edge deployment via Bernstein polynomial activations. **Bern2Edge** distills large pretrained DNNs into compact BNNs that preserve accuracy while exposing two deployment paths: direct LUT-based FPGA synthesis and geometry-aligned symbolic rule extraction. Across all evaluated architectures and datasets, BNNs outperform ReLU under compression, achieving 97.3–99.8% latency reduction and substantial resource savings over a quantized teacher within 0.5 pp accuracy. The LUT-based path eliminates the activation train–deploy gap for exact, efficient hardware realization, while the geometric structure of Bernstein activations yields compact, interpretable rules with explicit input-space constraints.

REFERENCES

- [1] W. Jiang, L. Yang, E. H.-M. Sha, Q. Zhuge, S. Gu, S. Dasgupta, Y. Shi, and J. Hu, “Hardware/software co-exploration of neural architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 4805–4815, 2020.
- [2] J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran *et al.*, “Fast inference of deep neural networks in fpgas for particle physics,” *Journal of instrumentation*, vol. 13, no. 07, pp. P07 027–P07 027, 2018.
- [3] M. Blott, T. B. Preußer, N. J. Fraser, G. Gambardella, K. O’Brien, Y. Umuroglu, M. Leeser, and K. Vissers, “Finn-r: An end-to-end deep-learning framework for fast exploration of quantized neural networks,” *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 11, no. 3, pp. 1–23, 2018.
- [4] G. Abarajithan, Z. Ma, R. Munasinghe, F. Restuccia, and R. Kastner, “cg4ml: A hardware/software framework to implement neural networks for scientific edge computing,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 19, no. 2, pp. 1–33, 2026.
- [5] H. Khedr and Y. Shoukry, “Deepbern-nets: Taming the complexity of certifying neural networks using bernstein polynomial activations and precise bound propagation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 19, 2024, pp. 21 232–21 240.
- [6] R. T. Farouki, “The bernstein polynomial basis: A centennial retrospective,” *Computer Aided Geometric Design*, vol. 29, no. 6, pp. 379–419, 2012.
- [7] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [8] D. Hendrycks and K. Gimpel, “Gaussian error linear units (gelus),” *arXiv preprint arXiv:1606.08415*, 2016.
- [9] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for activation functions,” *arXiv preprint arXiv:1710.05941*, 2017.
- [10] K. Choi, S. Kim, J. Kim, and I.-C. Park, “Hardware-friendly approximation for swish activation and its implementation,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 10, pp. 4516–4520, 2024.
- [11] E. Wang, J. J. Davis, P. Y. Cheung, and G. A. Constantinides, “Lutnet: Rethinking inference in fpga soft logic,” in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2019, pp. 26–34.
- [12] Y. Umuroglu, Y. Akhauri, N. J. Fraser, and M. Blott, “Logicnets: Co-designed neural networks and circuits for extreme-throughput applications,” in *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*, 2020, pp. 291–297.
- [13] M. Andronic and G. A. Constantinides, “Polylut: learning piecewise polynomials for ultra-low latency fpga lut-based inference,” *arXiv preprint arXiv:2309.02334*, 2023.
- [14] B. Lou, R. Rademacher, D. Bolland, and P. H. Leong, “Polylut-add: Fpga-based lut inference with wide inputs,” in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2024, pp. 149–155.
- [15] D. Hoang, A. Gupta, and P. C. Harris, “Kanelé: Kolmogorov–arnold networks for efficient lut-based evaluation,” in *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2026, pp. 44–55.

TABLE XI: Distribution-shift robustness on ACS Income, training on CA-2018. ID = held-out CA-2018 test set; GEO-AVG and TEMP-AVG are means over shifted conditions; Δ = AVG-ID (pp). Means $\pm$ std over 5 seeds.

System	Metric	ID (CA-18)	Geographic shift				Temporal shift (CA)					
			GEO-AVG	Δ	MS	WY	WV	TEMP-AVG	Δ	2019	2021	2022
ReLU Teacher	Acc. (%)	81.28 $\pm$ 0.09	74.32 $\pm$ 0.64	-6.9	73.73	74.61	74.61	80.00 $\pm$ 0.09	-1.3	80.74	79.82	79.44
ReLU	Acc. (%)	80.55 $\pm$ 0.15	74.10 $\pm$ 0.76	-6.5	73.42	74.45	74.43	79.37 $\pm$ 0.19	-1.2	80.03	79.23	78.84
BNN	Acc. (%)	80.89 $\pm$ 0.29	74.45 $\pm$ 0.69	-6.4	73.79	74.77	74.79	79.81 $\pm$ 0.36	-1.1	80.49	79.67	79.26
Rules	Coverage (%)	90.09 $\pm$ 0.86	85.82 $\pm$ 1.38	-4.3	86.10	85.07	86.29	90.24 $\pm$ 0.78	+0.2	90.01	90.35	90.35
	Covered acc. (%)	80.29 $\pm$ 0.20	76.41 $\pm$ 1.07	-3.9	77.31	74.82	77.09	78.67 $\pm$ 0.42	-1.6	79.38	78.62	78.00
	Total acc. (%)	78.58 $\pm$ 0.12	73.15 $\pm$ 1.35	-5.4	73.41	72.44	73.61	77.24 $\pm$ 0.29	-1.3	77.78	77.24	76.70

TABLE XII: SST-2 accuracy, latency, and FPGA resource utilization for full 4-layer FFN substitution. FFN: FFN sublayers only; Full: complete encoder. Resource reductions are relative to TinyBERT4.

Scope	Metric	TinyBERT4	h=600		h=312	
			GeLU	Bern	GeLU	Bern
FFN	SST-2 Acc. (%)	90.37	90.02	90.48	89.11	90.02
	Lat. (cycles)	6,413,520	3,282,640 ($\downarrow$ 48.8%)	3,278,288 ($\downarrow$ 48.9%)	1,789,648 ($\downarrow$ 72.1%)	1,785,296 ($\downarrow$ 72.2%)
	DSP	121	121 (0.0%)	80 ($\downarrow$ 33.9%)	121 (0.0%)	80 ($\downarrow$ 33.9%)
	BRAM	155	82 ($\downarrow$ 47.1%)	128 ($\downarrow$ 17.4%)	80 ($\downarrow$ 48.4%)	108 ($\downarrow$ 30.3%)
	FF	17,746	19,580 ($\uparrow$ 10.3%)	15,654 ($\downarrow$ 11.8%)	19,528 ($\uparrow$ 10.0%)	15,568 ($\downarrow$ 12.3%)
	LUT	34,744	35,165 ($\uparrow$ 1.2%)	30,070 ($\downarrow$ 13.5%)	34,924 ($\uparrow$ 0.5%)	29,789 ($\downarrow$ 14.3%)
Full	Lat. (cycles)	7,591,584	4,460,704 ($\downarrow$ 41.2%)	4,456,352 ($\downarrow$ 41.3%)	2,967,712 ($\downarrow$ 60.9%)	2,963,360 ($\downarrow$ 61.0%)
	DSP	319	319 (0.0%)	278 ($\downarrow$ 12.9%)	319 (0.0%)	278 ($\downarrow$ 12.9%)
	BRAM	164	91 ($\downarrow$ 44.5%)	137 ($\downarrow$ 16.5%)	89 ($\downarrow$ 45.7%)	117 ($\downarrow$ 28.7%)
	FF	40,302	42,049 ($\uparrow$ 4.3%)	38,210 ($\downarrow$ 5.2%)	41,997 ($\uparrow$ 4.2%)	38,037 ($\downarrow$ 5.6%)
	LUT	81,034	81,378 ($\uparrow$ 0.4%)	76,360 ($\downarrow$ 5.8%)	81,137 ($\uparrow$ 0.1%)	76,002 ($\downarrow$ 6.2%)

- [16] M. Goyal, R. Goyal, and B. Lall, "Improved polynomial neural networks with normalised activations," in *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2020, pp. 1–8.
- [17] I. Albool, M. G. El-Din, S. Elmalaki, and Y. Shoukry, "From dead neurons to deep approximators: Deep bernstein networks as a provable alternative to residual layers," *arXiv preprint arXiv:2602.04264*, 2026.
- [18] M. Craven and J. Shavlik, "Extracting tree-structured representations of trained networks," *Advances in neural information processing systems*, vol. 8, 1995.
- [19] J. R. Zilke, E. Loza Mencía, and F. Janssen, "Deepred-rule extraction from deep neural networks," in *International conference on discovery science*. Springer, 2016, pp. 457–473.
- [20] M. E. Zarlenga, Z. Shams, and M. Jamnik, "Efficient compositional rule extraction for deep neural networks," *arXiv preprint arXiv:2111.12628*, 2021.
- [21] C.-M. Pan, S. Elmalaki, Y. Shoukry, and S. Huang, "NeuSym-HLS: Learning-Driven Symbolic Distillation in High-Level Synthesis of Hardware Accelerators," Presented at the NeurIPS Workshop on Machine Learning for Systems (MLForSys), San Diego, CA, USA, Dec. 2025, available: <https://neurips.cc/virtual/2025/loc/san-diego/129116>.
- [22] S. Thrun, "Extracting rules from artificial neural networks with distributed representations," *Advances in neural information processing systems*, vol. 7, 1994.
- [23] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," *Advances in neural information processing systems*, vol. 34, pp. 18 932–18 943, 2021.
- [24] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [25] J. Blackard, "Covertypes," UCI Machine Learning Repository, 1998, DOI: <https://doi.org/10.24432/C50K5N>.
- [26] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, DOI: <https://doi.org/10.24432/C5XW20>.
- [27] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
- [28] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for exotic particles in high-energy physics with deep learning," *Nature communications*, vol. 5, no. 1, p. 4308, 2014.
- [29] J. Vanschoren, J. N. Van Rijn, B. Bischl, and L. Torgo, "Openml: networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.
- [30] R. Bock, "MAGIC Gamma Telescope," UCI Machine Learning Repository, 2004, DOI: <https://doi.org/10.24432/C52C8B>.
- [31] F. Ding, M. Hardt, J. Miller, and L. Schmidt, "Retiring adult: New datasets for fair machine learning," in *NeurIPS*, 2021.
- [32] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proceedings of the 2013 conference on empirical methods in natural language processing*, 2013, pp. 1631–1642.
- [33] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "Tinybert: Distilling bert for natural language understanding," in *Findings of the association for computational linguistics: EMNLP 2020*, 2020, pp. 4163–4174.
- [34] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [35] Z. Shams, B. Dimanov, S. Kola, N. Simidjievski, H. A. Terre, P. Scherer, U. Matjašec, J. Abraham, P. Liò, and M. Jamnik, "Rem: an integrative rule extraction methodology for explainable data analysis in healthcare," *medRxiv*, pp. 2021–01, 2021.
- [36] Advanced Micro Devices, Inc., *7 Series FPGAs Data Sheet: Overview*, Advanced Micro Devices, Inc., San Jose, CA, USA, 2020.
- [37] "IEC 61298-2: Process measurement and control devices — general methods and procedures for evaluating performance — Part 2: Tests under reference conditions," International Electrotechnical Commission, Geneva, Switzerland, International Standard IEC 61298-2, 2008.
- [38] K. Xu, Z. Shi, H. Zhang, Y. Wang, K.-W. Chang, M. Huang, G. Katz, and J. Z. Kolter, "Automatic perturbation analysis for scalable certified robustness and beyond," *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [39] B. Kim, M. Wattenberg, J. Gilmer, R. Caruana, E. Wiewiora, F. Viegas, and R. Sayres, "Interpretability beyond classification: Quantitative testing with concept activation vectors (TCAV)," in *International Conference on Machine Learning (ICML)*, 2018.
- [40] G. G. Chrysos, S. Moschoglou, G. Bouritsas, Y. Panagakis, J. Deng, and S. Zafeiriou, "P-nets: Deep polynomial neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 7325–7335.